

Module 13: Interview Cheatsheet & Summary

This cheatsheet serves as a high-density revision reference for Autonomous AI Agents and Multi-Agent system design, compiling equations, architectural patterns, complexity matrices, and high-yield interview questions.

1. 8 Must-Know Agent Formulas & Rules

1. **UCT (Upper Confidence bounds for Trees) in LATS:**

$$a_{\text{best}} = \arg \max_a \left(\frac{Q(s, a)}{N(s, a)} + c \cdot \sqrt{\frac{\ln N(s)}{N(s, a)}} \right)$$

2. Guides the selection phase during search tree traversals.

3. **Self-Consistency Voting (Majority Decision Rule):**

$$\hat{y} = \arg \max_v \sum_{k=1}^K \mathbb{I}(y_k = v)$$

4. Aggregates outputs from \$K\$ sampling paths using temperature \$T > 0\$.

5. **Execution Loop Budget Cost:**
$$\text{Total Cost} = \sum_{t=1}^T \left(\text{Input Tokens}_t \cdot \text{Cost}_{\text{in}} + \text{Output Tokens}_t \cdot \text{Cost}_{\text{out}} \right)$$

6. Tracks total session expenditure across all loops.

7. **Agent Confidence Gating Decision Rule:**

$$\text{Route} = \begin{cases} \text{Execute Tool Autonomously} & \text{if } C \geq C_{\text{threshold}} \\ \text{Escalate to Human (HITL)} & \text{if } C < C_{\text{threshold}} \end{cases}$$

8. **ReAct Delimiter Formats Rule:**

9. Every ReAct sequence must strictly emit: `Thought:` `$\rightarrow$` `Action:` `$\rightarrow$` `Observation:` `$\rightarrow$` `Reflect:` `$\rightarrow$` `Final Answer:`.

10. **Topological Sort Rule for DAG Schedulers:**

11. Tasks are queued for execution only when in-degree dependency count $\text{in_degree}(v) = 0$.

12. **Idempotency Gating Rule:**

13. Every write API trigger must contain a header payload `X-Idempotency-Key = UUID(session_id + turn_k)`.

14. **Loop Guard Boundary Rule:**

15. Halt execution and alert if current action tuple matches any of the last \$k\$ signatures: $(A_t, \text{Args}_t) \in \{(A_{t-1}, \text{Args}_{t-1}), \dots, (A_{t-k}, \text{Args}_{t-k})\}$

2. 8 Core Agent Architectures

Pattern	Key Characteristic	Primary Use Case
1. Single ReAct Loop	Thought/Action/Observation iterations	Simple API routing, research
2. Supervisor	Central dispatcher routing workers	Task delegation, multi-domain

3. Hierarchical	Nested supervisor organization tree	Large document/code gen
4. Sequential Chain	Linear state passage (A -> B -> C)	Pipelines, write-review
5. Debate	Conflicting agent prompts arguing	Fact check, safety auditor
6. Planner-Executor	Planner writes DAG, Executor calls tools	Code compilation, workflows
7. Blackboard	Shared asynchronous publish database	Event-triggered scaling
8. Reflexion	Actor-critic self-correction loop	Self-correcting coders

3. Time & Memory Complexity Matrix

Module	Algorithm / Loop Phase	Time Complexity	Memory/Space Complexity
01	Agent Execution step	$O(T \cdot d^2)$	$O(T \cdot N)$ context RAM
02	Observe-Reason-Act loop	$O(T \cdot (P \cdot d + d^2))$	$O(N_s + N_h)$ state RAM
03	Self-Consistency Sample	$O(K \cdot N_{len})$	$O(K \cdot N_{len})$ generation buffers
04	Tree of Thoughts (ToT)	$O(b^d \cdot N_{tokens})$	$O(b^d)$ node states in memory
05	DAG Scheduler Traversal	$O(V + E)$	$O(V + E)$ dependency RAM
06	Vector Memory Retrieval	$O(d \cdot \log M)$ (HNSW)	$O(M \cdot d)$ vector DB storage
07	Parameter Schema Validation	$O(P)$ check time	$O(P \cdot d)$ parameter metadata
08	Supervisor Routing	$O(C \cdot T \cdot d^2)$	$O(\sum N_i)$ worker context RAM
09	Confidence HITL Gating	$O(1)$ constant check	$O(1)$ check variables
10	LLM-as-a-Judge Evaluation	$O(T_{tr} \cdot N_{tokens})$	$O(T_{tr} \cdot N_{tokens})$ log storage
11	Sandboxed Execution	$O(T \cdot (t_{api} + t_{inf}))$	$O(N_c)$ temporary code sandbox RAM
12	Loop Guard History search	$O(T_{history})$ signature check	$O(T_{history})$ signature strings

Complexity Variable Legend:

- T : Number of execution steps/iterations in the loop.
- d : Underlying model embedding dimension.
- N : Context window token size.
- P : Count of parameters defined in target schema.
- K : Number of parallel sample paths generated.
- b : Tree branching factor.
- M : Number of documents/vectors stored in database.
- V : Number of vertex tasks in DAG.
- E : Number of dependency edges in DAG.

- \$C\$: Number of specialized worker agents.
 - \$T_{tr}\$: Number of execution steps in evaluated trajectory.
 - \$N_s\$: Size of stored system state parameters in bytes.
 - \$N_h\$: Dialog context window history size.
-

4. 30 High-Yield Interview Q&As

Category A: Agent Fundamentals & Architecture

1. *What is the difference between an LLM agent and a deterministic DAG workflow?*
2. Workflows follow pre-coded paths; agents dynamically select paths step-by-step using LLM reasoning.
3. *Describe the Level 4 vs. Level 5 autonomy boundary.*
4. Level 4 halts and escalates to human review on errors; Level 5 self-corrects and runs autonomously to completion.
5. *What is the purpose of the 'Thought' token generation step in ReAct loops?*
6. Acts as a scratchpad context, aligning variables and parameters to stabilize logic prior to invoking tools.
7. *How does the Blackboard pattern differ from the Supervisor pattern?*
8. Blackboard uses decoupled agents polling a shared DB; Supervisor uses a central orchestrator dispatching calls.
9. *Why is sandboxing necessary for coding assistants?*
10. Prevents LLM-generated code from executing malicious commands, deleting host directories, or exhausting memory resources.
11. *How do you solve context window drift over 40 turns?*
12. Summarize old turns, discard intermediate logs, and parse dialogue history into a structured key-value state store.

Category B: Reasoning & Planning

1. *How does Tree of Thoughts (ToT) solve the backtracking limitation of ReAct?*
2. Evaluates child thought branches and uses DFS/BFS to backtrack to parent nodes when paths score poorly.
3. *What is the exploration-exploitation trade-off in LATS?*
4. UCT balances exploiting high-scoring thought nodes vs. exploring nodes with low sample visit counts.
5. *What is Self-Consistency and why does temperature matter?*
6. Generates \$K\$ reasoning chains; majority votes on the answer. Temperature \$T > 0\$ is needed to generate diverse paths.
7. *How does Least-to-Most prompting improve performance on math tasks?*
 - Decomposes the query into sub-problems, solving them sequentially to prevent context attention dilution.
8. *Describe how dynamic replanning adapts to an API outage.*
 - The failed tool output is passed to the LLM as an observation, triggering it to choose alternative APIs or fallback scripts.
9. *How do you write a Topological Sort for a task DAG scheduler?*
 - Queue tasks with in-degree = 0. Upon execution success, decrement child in-degrees and queue new nodes.

Category C: Tool Calling & Memory

1. *What is function calling token masking?*
 - The server constrains output tokens to only match valid keys and characters of a target JSON schema.
2. *Explain the risk of parameter type hallucination.*
 - LLM generates syntactically valid parameters that fail tool execution (e.g. passing a string username to an ID field).
3. *Why is idempotency critical for write-based tools?*

- Network timeouts can cause retries; unique idempotency keys prevent duplicate payments or record creations.
- 4. *Compare Episodic vs. Semantic memory in agents.*
 - Episodic stores logs of past execution runs; Semantic stores static knowledge bases and document embedding indices.
- 5. *What is memory compaction?*
 - Summarizing conversation logs or consolidating facts into an entity graph database.
- 6. *How does an LRU cache optimize long-term memory retrieval?*
 - Evicts least-recently queried vector embeddings to conserve local memory capacity.

Category D: Multi-Agent & HITL

1. *Why does division of labor work better in multi-agent systems?*
 - Restricts context scope per agent, reducing vocabulary collision and improving reasoning accuracy.
2. *Explain the supervisor-worker communication topology.*
 - Workers only talk to the central supervisor, who acts as a routing node, preventing $O(N^2)$ message inflation.
3. *Describe the Debate pattern and when to use it.*
 - Agents generate opposing arguments evaluated by a Judge model. Best for risk analysis and safety checks.
4. *How do you establish a HITL confidence gate?*
 - If task routing classification confidence score is below 0.85, suspend the loop and flag for manual override.
5. *What is escalation routing?*
 - Gating high-risk actions (e.g. paying > \$500) behind manual human confirmation screens.
6. *How do you represent multi-agent states in stateful engines?*
 - Save states in a central database (PostgreSQL) linked by `session_id`, loading specific sub-states for each worker.

Category E: Production Design, Security & Failures

1. *Describe indirect prompt injection.*
 - Malicious instructions embedded in scraped websites or parsed emails hijack the agent's main system prompt.
2. *How do you defend against infinite execution loops?*
 - Enforce hard step count limits (≤ 10) and action signature duplicate checks.
3. *What is a cascading failure?*
 - An error in an early task observation corrupts the context history, causing all subsequent steps to fail.
4. *How do you perform a trajectory replay?*
 - Serialize the exact state, system prompt, and context logs of a failed run, and execute locally at temperature 0.0.
5. *Why are rate limits needed for tool executors?*
 - Prevents runaway agent loops from making millions of API hits and incurring high token costs.
6. *How do you design a stateless agent worker architecture?*
 - Use message queues (RabbitMQ) to route task logs, and load/save agent state parameters from PostgreSQL at each step.